STA 3402 – Machine Learning & Neural Networks

Lecture 4 – Cluster Analysis: k-means, Hierarchical (Agglomerative & Divisive) Clustering

Lecturer: Dr. Sameera Viswakula, Department of Statistics, University of Colombo

Full study notes from the lecture recording, the "Clustering Algorithms" slides and the R code (simple English).

All outputs and plots in these notes come from actually running the R code.

Contents

1. What is clustering and why do we use it?
2. First practical: k-means on the iris data
3. Within and between sum of squares – how good is a clustering?
4. The big picture: types of clustering
5. The milk example (slides)
6. Distance: Euclidean and Manhattan
7. Hierarchical clustering: steps and linkage methods
8. Single linkage example with five genes (g1–g5)
9. Example 2: data from three normal populations
10. k-means clustering – the theory
11. Example 3: k-means and dendrograms on simulated data
12. Divisive clustering with DIANA
13. Homework (due 05.01.2026)
14. Extra practice: fish data and penguins data
15. Summary

1. What is clustering and why do we use it?

> **Definition (slides)**
> **Clustering** is the process of grouping several objects into a number of groups, or **clusters**.
> **Goal:** objects in the same cluster are **more similar to one another** than they are to objects in other clusters.

- Imagine a scatter plot of variable 1 and variable 2. We may see some **groups**.
- Instead of fitting **one** regression model to the whole data set, we can fit a **separate model for each cluster** – this is often more effective.
- When we cluster, we don't know in advance **which features** create the groups. After clustering we must **dig deeper**: look at the observations in cluster 1 and cluster 2 and find the key features (or combinations) that make them different – e.g. gender, where

people live, demographic or social factors. Sometimes the clusters cannot be described in a simple way.

- Clustering is an **initial or intermediate step**: **pre-process the data → cluster → model**. We never stop after clustering.
- We group **observations** (rows), **not variables**.
- There is no label (no response variable), so clustering is **unsupervised learning**.

2. First practical: k-means on the iris data

Students named **k-means** as a clustering technique. "k-means" means we divide the data into a **pre-specified number of groups, k**. We can run it for k = 2, 3, 4, 5 … and decide on the number of clusters by looking at measures such as the within/between sum of squares, a scree-like (elbow) plot or a silhouette plot.

Exercise: load the iris data set, apply k-means clustering, and say how many clusters you see. (If you don't know the function, open its help page `?kmeans` and look at the required arguments.)

2.1 Load and look at the data

```
# STA 3402 – Clustering: K-means clustering
data(iris)
dim(iris)        # 150 obs and 5 variables
head(iris)
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa

2.2 Why only quantitative variables?

Just like PCA (dimension reduction), basic clustering uses **only quantitative data**. Why? Clustering is based on **distances between observations**: points that are close are put in the same cluster, far-away points are left out. To compute a distance (e.g. Euclidean distance – "get the x difference and y difference, square, add, take the square root", like in school) we need **numbers**.

Person	Age	Gender
1	37	Male
2	48	Female
3	90	Male

How far is a 37-year-old male from a 48-year-old female? We can measure the age difference, but **we cannot measure the distance between "male" and "female"**. So for this basic clustering we keep only the quantitative variables. (Categorical data needs other methods – e.g. MCA/FAMD coordinates, as in the last lecture.)

2.3 Copy the data and keep the numeric columns

```
# make a copy of the dataset
iris_data = iris
# extract only the quantitative variables
str(iris_data)                # Species is a Factor (the 5th variable)
iris_quants = iris_data[-5]    # all columns except the 5th
```

- Always work on a **copy**. If you change the original, you may not have all the variables for the rest of the analysis (or for an exam question on the same data).
- `str()` shows that Species is a **factor**. Here the data set is defined properly; otherwise you must read the variable descriptions to find the categorical variables.
- `iris_data[-5]` removes column 5 (a data frame can be indexed by columns directly).

2.4 Run k-means for k = 2, 3, 4

```
# k-means
kmeans(iris_quants, 2)   # 77.6 %
kmeans(iris_quants, 3)   # 88.4 %
table(iris_data$Species)
kmeans(iris_quants, 4)   # 89.5 %
```

We do **not** need to compute a distance matrix – `kmeans()` does everything. We only give the numeric data and the number of clusters (k = 1 is pointless: why cluster at all?).

Output for k = 2:

```
K-means clustering with 2 clusters of sizes 53, 97
Cluster means:
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1     5.005660    3.369811     1.560377    0.290566
2     6.301031    2.886598     4.958763    1.695876
Clustering vector:  1 1 1 1 1 ... 2 2 2 ...
Within cluster sum of squares by cluster:
[1]  28.55208 123.79588
 (between_SS / total_SS =  77.6 %)
Available components:
"cluster" "centers" "totss" "withinss" "tot.withinss" "betweenss" "size" "iter" "ifault"
```

Part of output	Meaning
sizes	Number of observations in each cluster
Cluster means (`$centers`)	The mean of each variable in each cluster – the cluster "centre"
Clustering vector (`$cluster`)	Which cluster each observation belongs to
Within cluster SS (`$withinss`)	How spread out each cluster is (small = tight cluster)
between_SS / total_SS	% of the total variation explained by the separation between clusters (closer to 100% = better)

k	between_SS / total_SS (lecture)	With `nstart = 25`
2	77.6%	77.6%
3	88.4%	88.4%
4	89.5%	91.6%

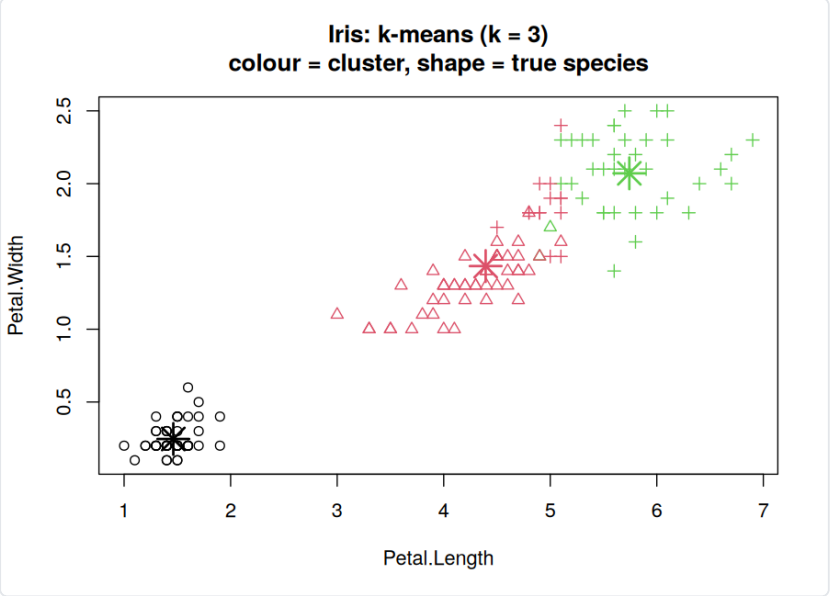
> **Why can your numbers be different?**
> k-means starts from **random** centres, so each run can end in a different (sometimes worse) solution. For example, one run of `kmeans(iris_quants, 3)` gave only 79.0%. Use `set.seed(123)` for reproducible results and `nstart = 25` (try 25 random starts, keep the best): `kmeans(iris_quants, 3, nstart = 25)`.

2.5 Compare with the real species

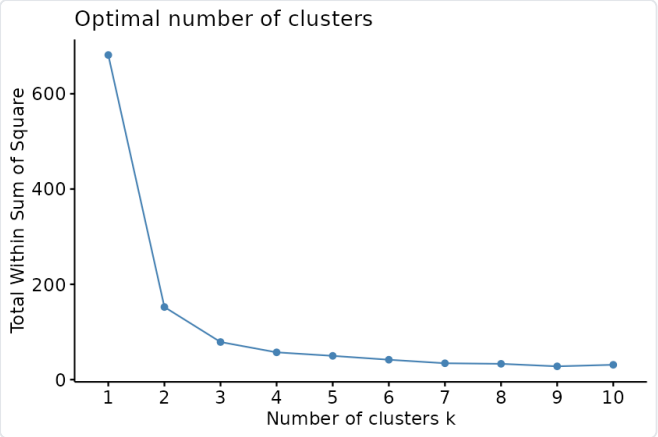
```
table(iris_data$Species)     # setosa 50, versicolor 50, virginica 50
```

The data set actually has **three flower species**: setosa, versicolor and virginica. We removed Species before clustering, so there was **no label** – that is why this is unsupervised learning. We use Species only afterwards, to check the result:

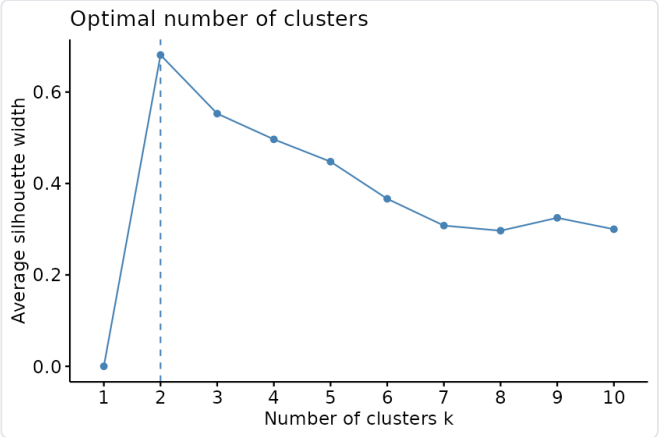
Cluster (k = 3)	setosa	versicolor	virginica
1	50	0	0
2	0	48	14
3	0	2	36



Setosa is perfectly separated; versicolor and virginica overlap a little



Elbow plot: `fviz_nbclust(iris_quants, kmeans, method = "wss")`



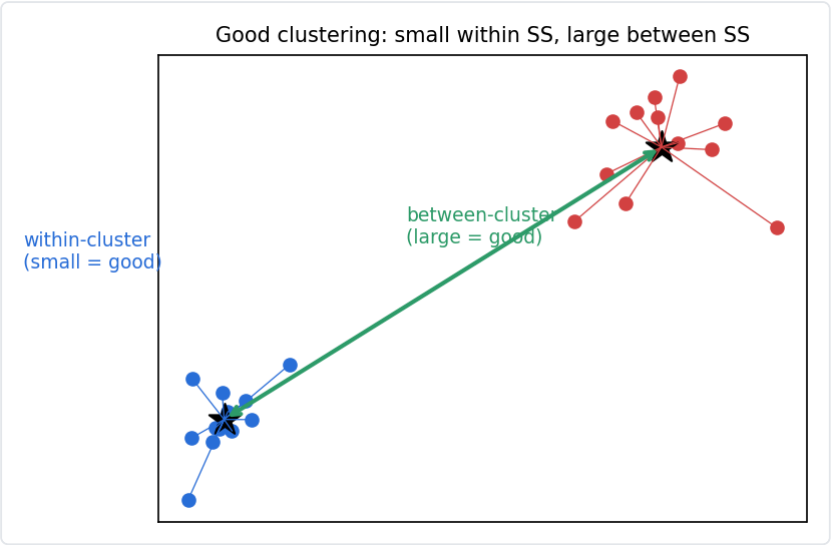
Silhouette plot: `method = "silhouette"`

k-means – advantage and disadvantage

Advantage: very straightforward and fast.

Disadvantage: we must decide the number of clusters **k before** running the algorithm, because k is one of the inputs.

3. Within and between sum of squares

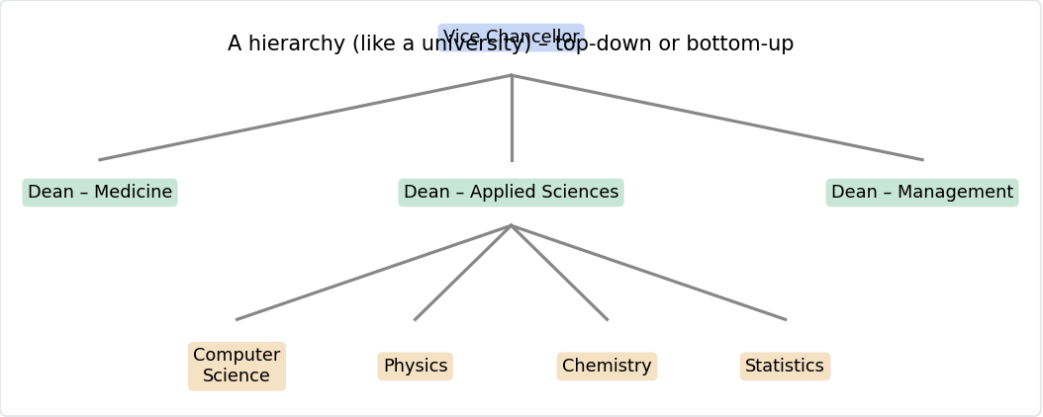


- If the data are clustered well, the distances **inside** each cluster are small → **within-cluster sum of squares is small**.
- The clusters are far from each other → **between-cluster sum of squares is large**.
- Total SS = Within SS + Between SS. So we look at **Between SS / Total SS**. The higher, the better (100% would be perfect). For iris with 2 clusters it is 77.6% – good.
- Adding more clusters always increases this ratio a little, so choose k where the gain becomes small (the **elbow**), and where the clusters make sense.

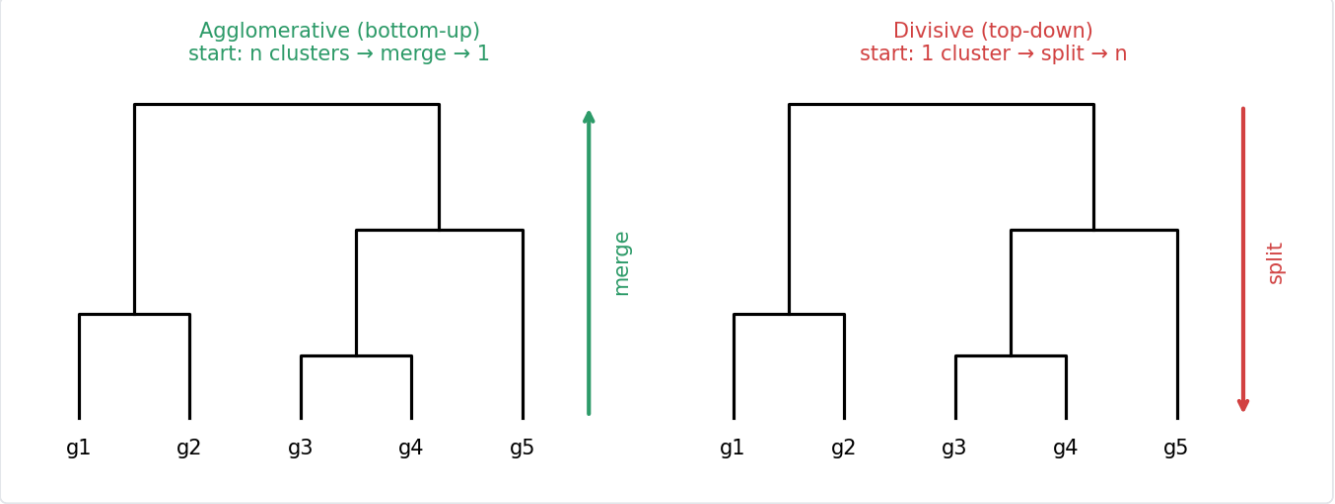
4. The big picture: types of clustering

Clustering has **no label information**, so it is **unsupervised** machine learning. There are two main types:

1. **k-means** (partitioning) clustering
2. **Hierarchical** clustering



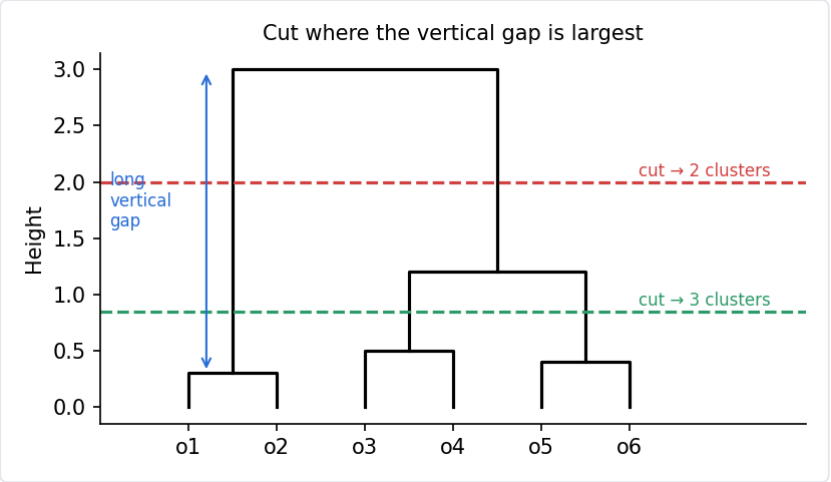
A **hierarchy** is like a university: the Vice Chancellor at the top, then the faculties with their Deans (Medicine, Applied Sciences, Management…), then the departments (Computer Science, Physics, Chemistry, Statistics…). You can read it from **top to bottom** or from **bottom to top**.



	Divisive clustering	**Agglomerative clustering**
Direction	**Top-down**	**Bottom-up**
Start	One single cluster with all observations (the root)	Each observation is its own cluster (n clusters, the leaves)
Process	Divide the most different (heterogeneous) cluster, based on some characteristics	Merge (combine) the most similar clusters
End	Every observation in its own cluster	One single big cluster
R	`diana()` (cluster package)	`hclust()` (stats package)

The tree diagram of a hierarchical clustering is called a **dendrogram**. This module focuses mainly on agglomerative clustering, but divisive clustering is also covered.

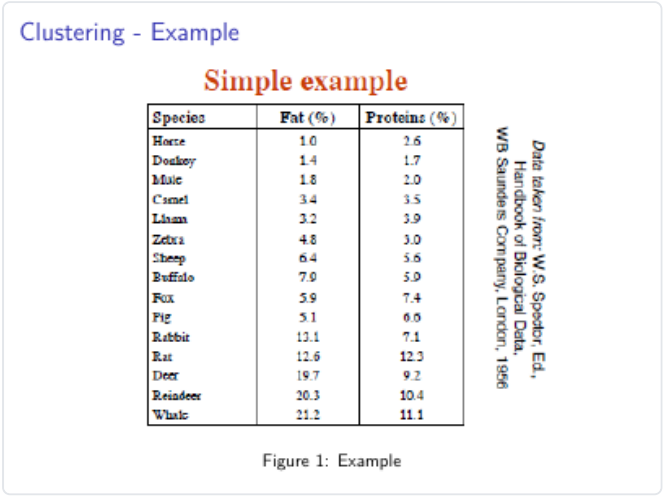
How many clusters from a dendrogram?



Draw a **horizontal line** across the dendrogram where the **vertical distance (gap) is relatively large** (not where it is very small). Then **count how many vertical branches the line cuts**. That number is the proposed number of clusters.

Note: so far we cluster only quantitative data (as with PCA last time). Categorical variables need other methods; the lecturer will come back to this.

5. The milk example (slides)


Clustering - Example

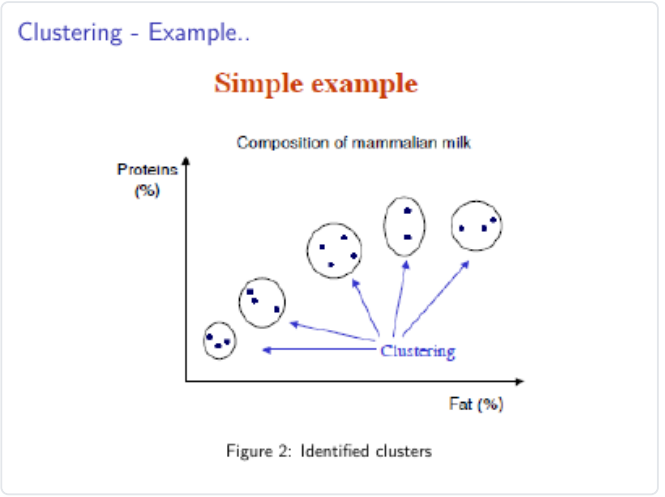
Simple example

Species	Fat (%)	Proteins (%)
Horse	1.0	2.6
Donkey	1.4	1.7
Mule	1.8	2.0
Camel	3.4	3.5
Llama	3.2	3.9
Zebra	4.8	3.0
Sheep	6.4	5.6
Buffalo	7.9	5.9
Fox	5.9	7.4
Pig	5.1	6.0
Rabbit	13.1	7.1
Rat	12.6	12.3
Deer	19.7	9.2
Reindeer	20.3	10.4
Whale	21.2	11.1

Data taken from: W.S. Spector, Ed., Handbook of Biological Data, WB Saunders Company, London, 1956

Figure 1: Example

Figure 1: composition of mammalian milk


Clustering - Example..

Simple example

Composition of mammalian milk

Proteins (%)

Clustering

Fat (%)

Figure 2: Identified clusters

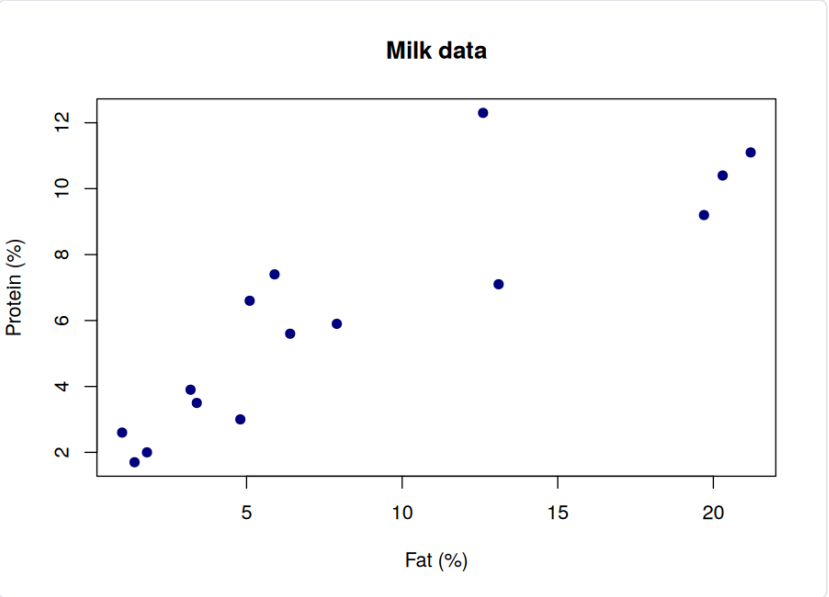
Figure 2: identified clusters

Mammals (animals that drink milk): horse, donkey, mule, camel, llama, zebra, sheep, buffalo, fox, rabbit, deer, reindeer, whale… For each species we know the **fat %** and **protein %** of its milk – two variables. From the plot you can see about **five clusters**.

The lecturer's tip: you can take a **screenshot** of a table, ask an AI tool to extract it to a CSV file, then load it into R. Or type the data directly:

```
milk_data = data.frame(
  Fat     = c(1,1.4,1.8,3.4,3.2,4.8,6.4,7.9,5.9,5.1,13.1,12.6,19.7,20.3,21.2),
  Protein = c(2.6,1.7,2.0,3.5,3.9,3,5.6,5.9,7.4,6.6,7.1,12.3,9.2,10.4,11.1))

plot(y = milk_data$Protein, x = milk_data$Fat)
```



> **Error: "figure margins too large"**
> `Error in plot.new(): figure margins too large` means the **Plots pane in RStudio is too small**. Drag the pane border to make it bigger (extend the lower panel), then run `plot()` again. (Other fixes: `dev.off()` to reset the device, or smaller margins with `par(mar = c(4,4,1,1))`.)

Popular clustering algorithms (slides): k-means, hierarchical clustering, self-organizing maps (SOM) and fuzzy clustering. We focus on k-means and hierarchical.

6. Distance

The concept of **distance** plays a crucial role in all types of cluster analysis. (Think of the distance between your campus in Mihintale and your home town – Galle, Matara, Kandy, Colombo, Kurunegala, Trincomalee…)

6.1 Distance between two numbers

$$d(a, b) = |a - b| = \sqrt{(a - b)^2}$$

It is **symmetric**: $d(a, b) = |a - b| = \sqrt{(a - b)^2} = |b - a| = d(b, a)$.

Example 1: $a = 1$, $b = 3 \rightarrow d(1, 3) = \sqrt{(1 - 3)^2} = \sqrt{4} = 2$, and $d(3, 1) = \sqrt{(3 - 1)^2} = \sqrt{4} = 2$. Same.

6.2 Euclidean distance between vectors

With gene expression values for several patients, we need distances between **vectors** $a = (a_1, \ldots, a_n)$ and $b = (b_1, \ldots, b_n)$. The most common one is the **Euclidean distance** = the **root of the sum of squared differences**:

$$d(a, b) = \sqrt{\left(\Sigma \, (a_i - b_i)^2 \right)}$$

Example 2: a = (1, 1), b = (4, 5):

$$d(a, b) = \sqrt{((1 - 4)^2 + (1 - 5)^2)} = \sqrt{(9 + 16)} = \sqrt{25} = 5$$

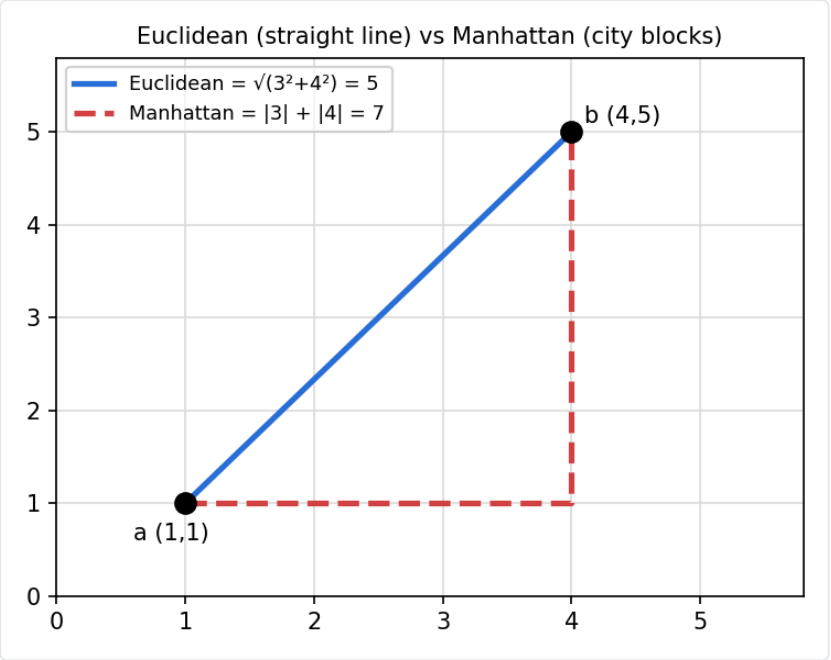
This is Pythagoras: the horizontal side is 3, the vertical side is 4, so the hypotenuse is $\sqrt{(3^2 + 4^2)} = 5$.

Example 3 (in R):

```
a <- c(1,1)
b <- c(4,5)
sqrt(sum((a-b)^2))        # 5

# your own function – works for any two vectors
euclid <- function(a, b) {
  return(sqrt(sum((a - b)^2)))
}
euclid(c(1,1), c(4,5))    # 5
```

6.3 Manhattan distance



- **Euclidean** = the straight line ("as the crow flies"), e.g. straight from the Mihintale town to the Faculty of Applied Sciences.
- **Manhattan** = you cannot go over the buildings (blocks between roads), so you go **around the blocks**: along one road, then another. On Manhattan Island in New York (Hudson River on one side, Brooklyn and New Jersey around it) the streets form a grid, and people give directions like "go two blocks down, turn left, go two more blocks".
- **Formula:** $d(a, b) = \Sigma \, |a_i - b_i|$. For a = (1,1), b = (4,5): |3| + |4| = 7.
- In R: `dist(x, method = "manhattan")`. Other options in `?dist`: "euclidean" (default), "maximum", "canberra", "binary", "minkowski".

7. Hierarchical clustering: steps and linkage

Two types (slides)

- **Agglomerative:** each observation starts as its own cluster (**leaf**); the most similar clusters are merged step by step until there is one big cluster (**root**). Bottom-up.
- **Divisive:** start with the root (all objects in one cluster); the most heterogeneous clusters are divided step by step until every observation is on its own. Top-down.

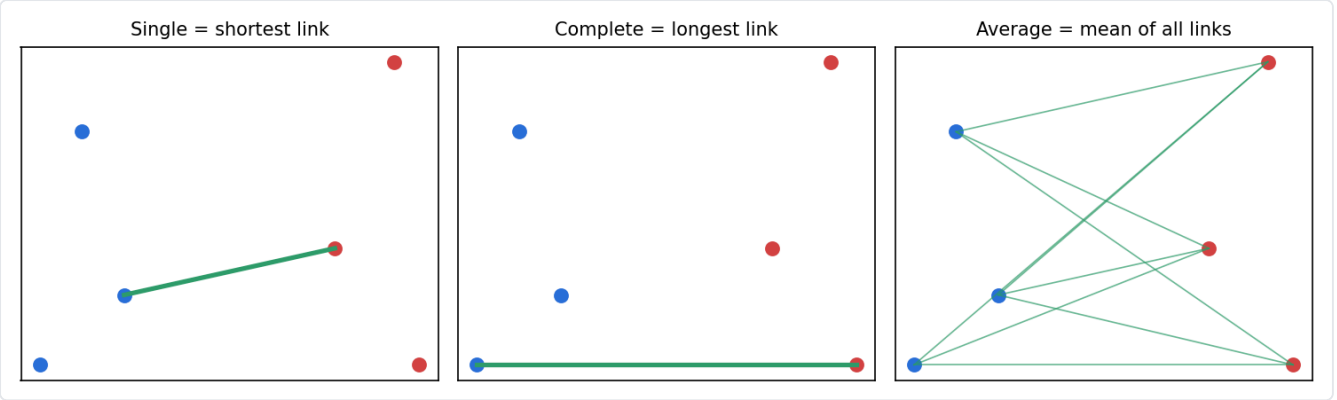
The result is a tree-based representation called a **dendrogram**.

7.1 Steps of agglomerative hierarchical clustering

1. **Prepare the data** (numeric, missing values handled, usually **scaled**).
2. **Compute the distance** (dissimilarity) between every pair of objects.
3. Use a **linkage function** to group objects into a hierarchical cluster tree, based on the distances. Objects/clusters that are close are linked together.
4. Decide where to **cut the tree** into clusters.

So there are **two ingredients**: the **distances** and the **linkage method**. Example: A–B distance = 2, A–C distance = 4. Which ones do we link, and how do we measure the distance from a *group* to another group? That is decided by the linkage.

7.2 Linkage methods – how to measure the distance between two clusters



Linkage	Distance between clusters I and J	Notes
Single	The **smallest** distance between a member of I and a member of J (nearest neighbours)	Can create long "chains"
Complete (maximum)	The **largest** distance between members (the two least similar members); merging minimises this maximum distance	Tends to give tight, spherical clusters. Default in `hclust()`
Average	The **average** of all distances between members of I and J	In between single and complete
Ward	Merge the pair that gives the smallest increase in the **within-cluster sum of squares**	A variance-minimising approach, similar to the k-means objective but done agglomeratively. In R: `"ward.D2"`

Single linkage formula (slides): $d(I, J) = \min \{ d(x_i, x_j) : x_i \text{ in } I \text{ and } x_j \text{ in } J \}$. So the distance between two clusters is the distance between their **nearest neighbours**.

Single linkage algorithm: start with as many clusters as data points → find the two nearest and merge them → find the next two nearest clusters and merge them → continue until all points are in one cluster.

> In this course you get some mathematical theory behind the methods – not as much matrix algebra as in Harvard or Stanford courses ("a diluted dose"), but the essentials. The subject is mostly practical.

8. Single linkage example with five genes

Five gene expressions from two persons: g1 = (1, 1), g2 = (1, 1.1), g3 = (3, 2), g4 = (3, 2.3), g5 = (5, 5). The values for each gene are coordinates on two axes p1 and p2.

8.1 Enter the data – rows must be the observations

```
# distance
?dist
df <- matrix(c(1,1,1,1.1,3,2,3,2.3,5,5),
             nrow = 2, byrow = FALSE)
df          # 2 x 5: rows = persons, columns = genes
t(df)       # 5 x 2: rows = genes (observations), columns = p1, p2
d = dist(t(df))
d
```

- A student entered the 10 numbers as one vector and made a matrix. With `nrow = 2, byrow = FALSE` R fills column by column, giving a 2 × 5 matrix (persons in rows).
- `dist()` computes distances **between the rows** (see `?dist`). We want distances between the **genes**, so the genes must be the rows.
- In statistics/data science a data set normally has **observations as rows** and **variables as columns**. So we **transpose** with `t(df)` – we don't change the data, only give the transposed version to `dist()`.

- The default method of `dist()` is **Euclidean**, so we don't need to specify it.

	g1	g2	g3	g4
g2	0.10			
g3	2.24	2.19		
g4	2.39	2.33	0.30	
g5	5.66	5.59	3.61	3.36

How to read it: the value in row g4, column g3 is d(g4, g3) = 0.30. The diagonal (a gene with itself) would be 0, and the upper half is the same as the lower half, so R shows only the lower triangle.

> The line `g(4,3) = 0.300000` in the R file is just a **note** about how to read the matrix – it is not R code. Put a `#` in front of it, otherwise R gives an error.

The slides do the same thing in one step, giving row and column names:

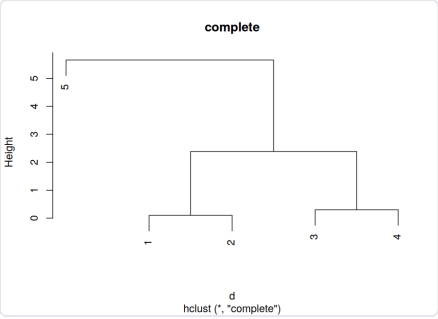
```
names <- list(c("g1","g2","g3","g4","g5"), c("p1","p2"))
sl.clus.dat <- matrix(c(1,1,1,1.1,3,2,3,2.3,5,5), ncol = 2,
                      byrow = TRUE, dimnames = names)
print(dist(sl.clus.dat, method = "euclidean"), digits = 3)
plot(sl.clus.dat, type = "n", xlim = c(0,6), ylim = c(0,6))
text(sl.clus.dat, labels = row.names(sl.clus.dat))
```

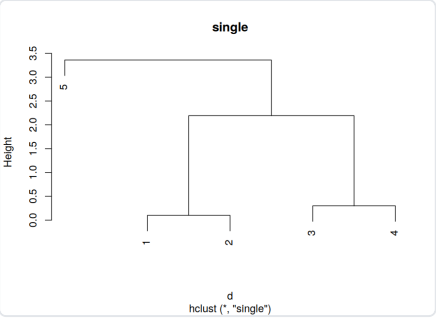
8.2 Hierarchical clustering with hclust()

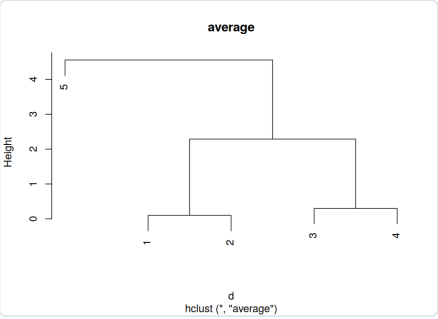
```
# hierarchical
?hclust                                          # stats package, default method = "complete"
hclust_out = hclust(d, method = "complete")      # give the DISTANCES, not the data
hclust_out
summary(hclust_out)                              # not very useful
plot(hclust_out)                                 # the dendrogram
```

- `hclust()` is in the **stats** package – no installation needed.
- Input: the **distance object** `d` (not the data set) and the **linkage** (`method`). The default is `"complete"`; writing it is optional but clear.
- Printing shows: Cluster method: complete, Distance: euclidean, Number of objects: 5.
- `summary()` only lists the parts (merge, height, order…), which are hard to read. The **plot** (dendrogram) is much more useful.

```
# how many clusters? why?
hclust_out2 = hclust(d, method = "single");   plot(hclust_out2)
hclust_out3 = hclust(d, method = "average");  plot(hclust_out3)
```







Merge	Single	Complete	Average
g1 + g2	0.10	0.10	0.10
g3 + g4	0.30	0.30	0.30
{g1,g2} + {g3,g4}	min = 2.19	max = 2.39	(2.24+2.19+2.39+2.33)/4 = 2.29
+ g5	min = 3.36	max = 5.66	(5.66+5.59+3.61+3.36)/4 = 4.55

With only 5 points the three linkages give the **same order**; only the heights differ. With other data sets they can change a lot.

> **How many clusters and why?**
> The biggest vertical gap is above the {g1,g2}+{g3,g4} merge (up to the g5 merge). A horizontal line there cuts **2 branches** → 2 clusters: {g1,g2,g3,g4} and {g5}. A line just above 0.30 cuts **3 branches** → {g1,g2}, {g3,g4}, {g5}. Both answers are acceptable – it is somewhat subjective; from the scatter plot 3 groups are clearly visible.

8.3 Single linkage – how it works (step by step)

1. At the start each point is a separate cluster.
2. The nearest two are g1 and g2 (d = 0.10). Merge them: I = {g1, g2}. In the tree this is the horizontal line at height 0.10. g3, g4, g5 are still separate.
3. The next smallest distance is d(g3, g4) = 0.30 → J = {g3, g4}, the line at height 0.30. Now there are three clusters: I, J and K = {g5}.
4. Distance I–J = smallest of d(g1,g3)=2.24, d(g1,g4)=2.39, d(g2,g3)=2.19, d(g2,g4)=2.33 = **2.19**. Distance J–K = min(3.61, 3.36) = 3.36. Distance I–K = min(5.66, 5.59) = 5.59. The smallest is I–J, so I and J merge at 2.19.
5. Finally the distance between {g1, g2, g3, g4} and g5 = d(g4, g5) = **3.36** → last merge.

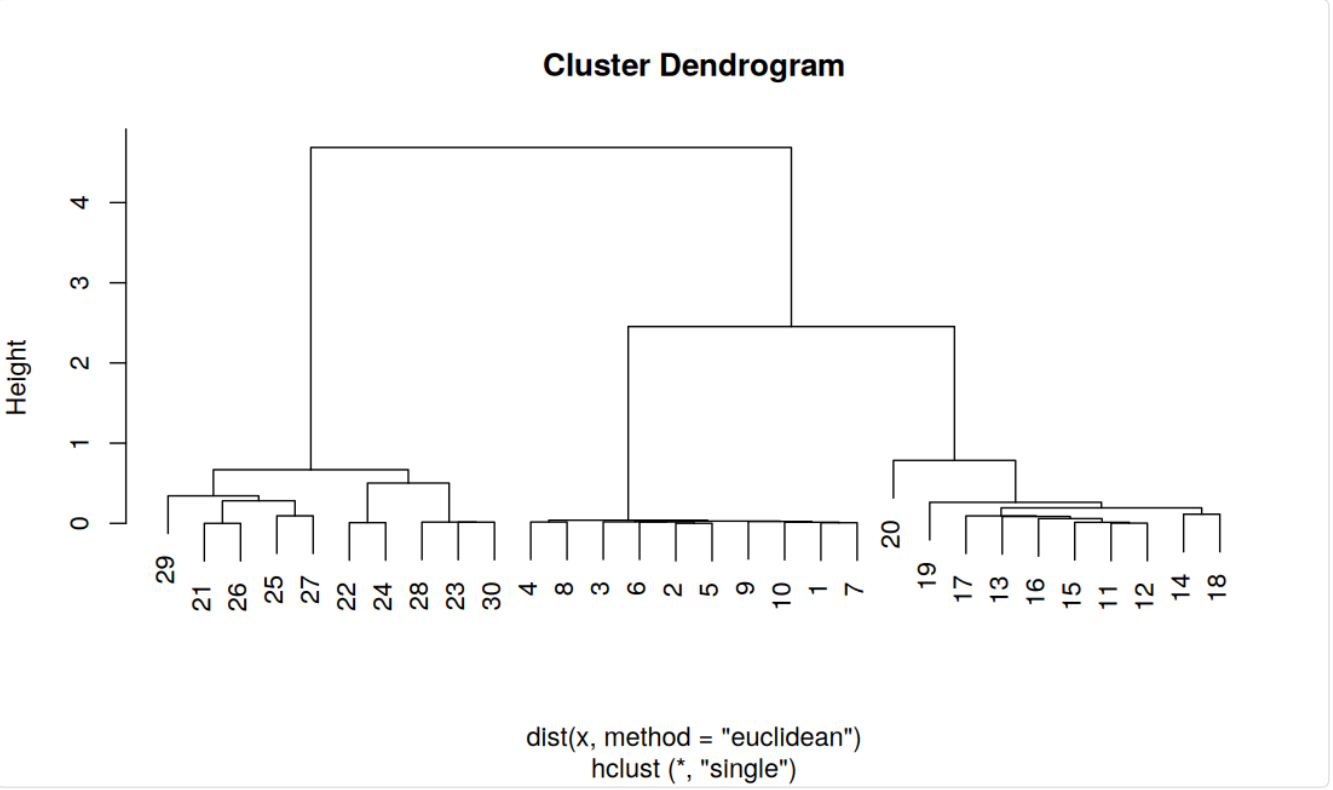
A typical R dendrogram shows the **distance method** and the **linkage method** under the plot (e.g. "dist(…, method = "euclidean")", "hclust (*, "single")").

9. Example 2: data from three normal populations

This example relates the **data generation process** to the cluster tree. Because the data are random, we set the seed to get the same numbers.

```
set.seed(100)
x = c(rnorm(10, 0, 0.1),     # 10 points, mean 0,  sd 0.1
      rnorm(10, 3, 0.5),     # 10 points, mean 3,  sd 0.5
      rnorm(10, 10, 1.0))    # 10 points, mean 10, sd 1
plot(hclust(dist(x, method = "euclidean"), method = "single"))
# OR store it first
sl.out = hclust(dist(x, method = "euclidean"), method = "single")
plot(sl.out)
```

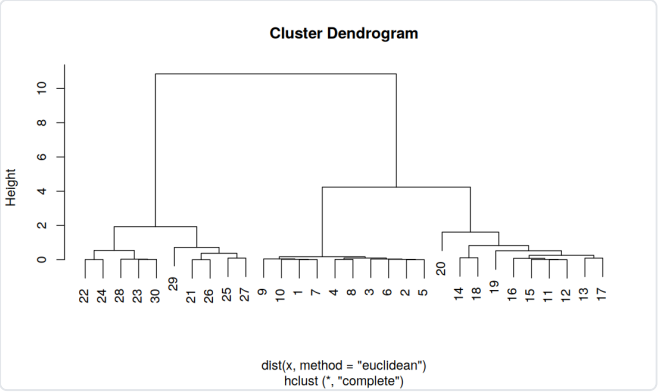
- `rnorm(n, mean, sd)` generates n random numbers from a normal distribution. The 30 values come from three different groups, so we expect **3 clusters**.
- Observations 1–10 come from group 1, 11–20 from group 2, 21–30 from group 3.



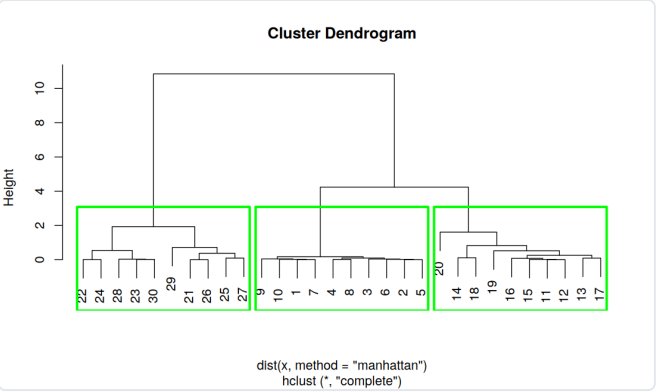
Single linkage, Euclidean distance

Look at the **largest vertical distances** and draw a horizontal line. A line high up gives **2 clusters**; a line lower gives **3 clusters**. Both answers are correct here. In practice it depends on the **domain**: if you work with a biologist or an economist, they check whether the clusters are meaningful.

```
plot(hclust(dist(x, method = "euclidean"), method = "complete"))   # change linkage
plot(hclust(dist(x, method = "manhattan"), method = "complete"))   # change distance
clm.out = hclust(dist(x, method = "manhattan"), method = "complete")
plot(clm.out)
rect.hclust(clm.out, k = 3)                       # boxes around 3 clusters
rect.hclust(clm.out, k = 3, border = "green")     # change the colour
```



Complete linkage – slightly different



Manhattan + complete, `rect.hclust(k = 3)`

- Changing the **linkage** or the **distance** changes the tree. Try different combinations to find the best cluster formation.
- `rect.hclust(tree, k = 3)` draws rectangles around the k clusters. `border` sets the colour.
- In class the lecturer tried to make the lines thicker. `rect.hclust()` has **no lwd argument** (its arguments are tree, k, which, x, h, border, cluster). A way that works: `op <- par(lwd = 3); rect.hclust(clm.out, k = 3, border = "green"); par(op)`.
- With 3 clusters, observations 1–10, 11–20 and 21–30 fall into three separate clusters – exactly how the data were generated.

10. k-means clustering – the theory

k-means is popular in bioinformatics and many other fields. It **minimises the within-cluster sum of squares** over K clusters. Given data points $x_1, \ldots, x_n$, it minimises

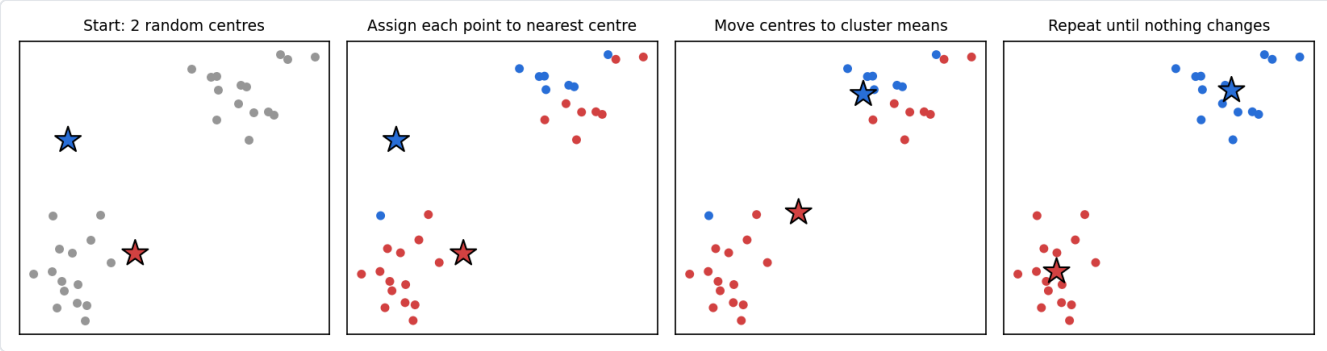
$$\Sigma_{k=1}^{K} \Sigma_{i \in I_k} d^2(x_i, a_k)$$

over all possible centre points $a_1, \ldots, a_K$.

- Distances can be positive or negative differences, so we use **squared** distances (no signs).
- Two indices: for cluster k = 1, add $d^2(x, a_1)$ for every point in cluster 1 (e.g. all 15 points). Then do the same for cluster 2, etc. Adding everything gives the **total within SS**.

The algorithm (Hartigan & Wong, 1979)

1. Split the data into K initial clusters, at random or with some heuristic.
2. **Step 1:** compute the cluster means.
3. **Step 2:** make a new partition by assigning each point to the **closest cluster mean**.
4. Repeat steps 1 and 2 until **convergence** – when no data point changes cluster any more.



11. Example 3: k-means and dendrograms on simulated data

We simulate gene expressions for two persons from two normal populations: 50 from N(0, 0.5) and 50 from N(2, 0.5). They are put in two 50 × 2 matrices placed one above the other → 100 data points. Then we run 2-means.

```
set.seed(100)
data <- rbind(matrix(rnorm(100, 0, 0.5), ncol = 2),
              matrix(rnorm(100, 2, 0.5), ncol = 2))
cl <- kmeans(data, 2)
cl
```

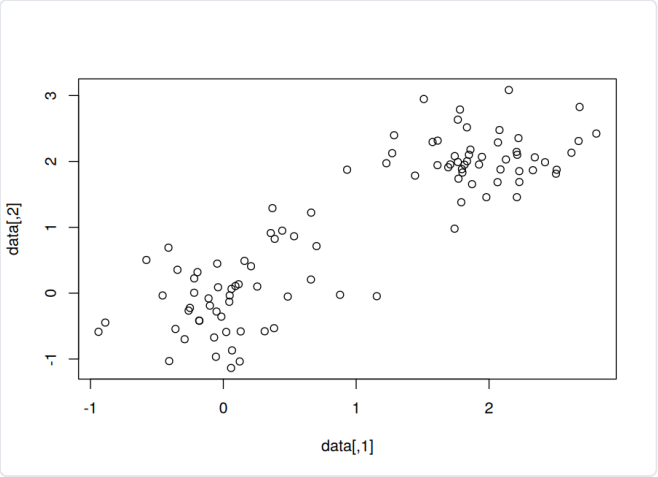
- `rnorm(100, 0, 0.5)` → 100 numbers; `matrix(..., ncol = 2)` → 50 rows × 2 columns.
- `rbind()` = **row bind**: puts the second matrix **below** the first → a 100 × 2 matrix.
- For k-means no distance calculation is needed – just the data and k.
- (The slides use `rnorm(50, …)`, which gives 25 + 25 = 50 points and 81.2%; the R file uses 100 + 100 numbers = 100 points.)

```
K-means clustering with 2 clusters of sizes 50, 50
Cluster means:
        [,1]        [,2]
1 1.95010671  2.06103413
2 0.04075447 -0.03784191
Clustering vector: 2 2 2 ... 2 1 1 1 ... 1
Within cluster sum of squares by cluster:
[1] 15.38181 25.63134
 (between_SS / total_SS =  83.1 %)
```

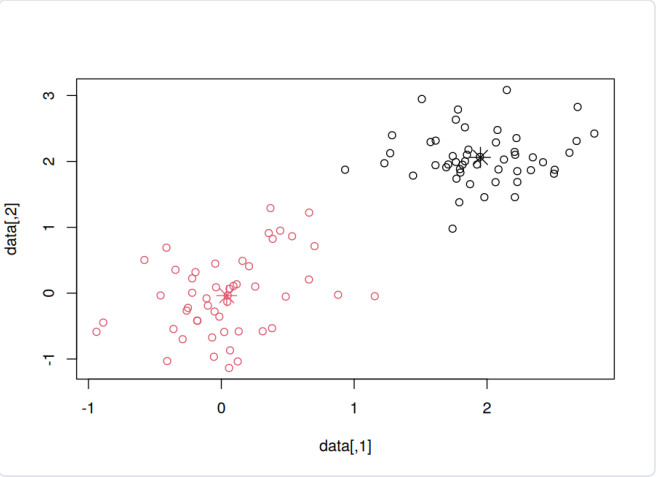
(In class, without the same seed, the lecturer got 79.2%.) With two variables, each cluster mean has two coordinates (x, y). Cluster **labels are arbitrary**: observations 1–50 form one cluster and 51–100 the other; which one is called "1" does not matter.

```
plot(data)                                          # raw data
plot(data, col = cl$cluster)                        # colour = cluster label
points(cl$centers, col = 1:2, pch = 8, cex = 2)     # add the centres
```

- `col = cl$cluster`: the cluster labels (1, 2) are used as colours (1 = black, 2 = red). You can also give colours like `c("red","black")[cl$cluster]`.
- `points()` adds points to an existing plot. `pch` = plotting character (8 = a star; try 2 = triangle, 19 = solid dot). `cex` = size.



plot(data)

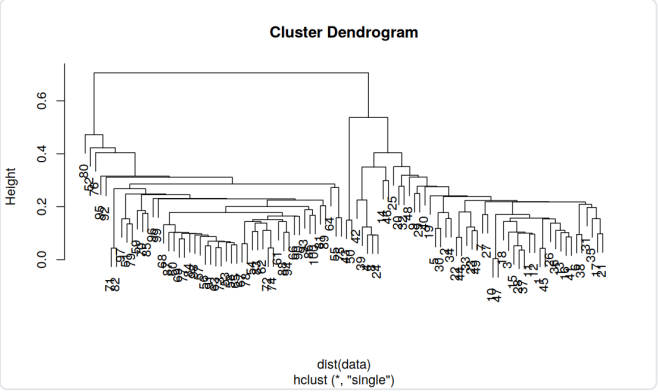


Coloured by cluster, stars = centres

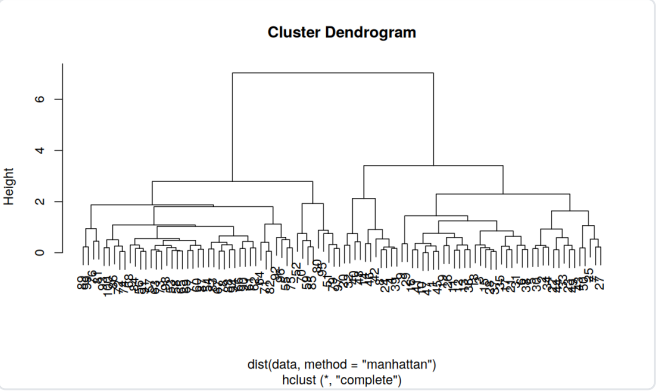
Exercise: obtain the dendrogram and identify the number of clusters

```
# obtain the dendrogram
d_data = dist(data)
hclust_out4 = hclust(d_data, method = "single")
plot(hclust_out4)

d_data = dist(data, method = "manhattan")
hclust_out4 = hclust(d_data, method = "complete")
plot(hclust_out4)
```



Euclidean + single: messy ("chaining")



Manhattan + complete: much clearer

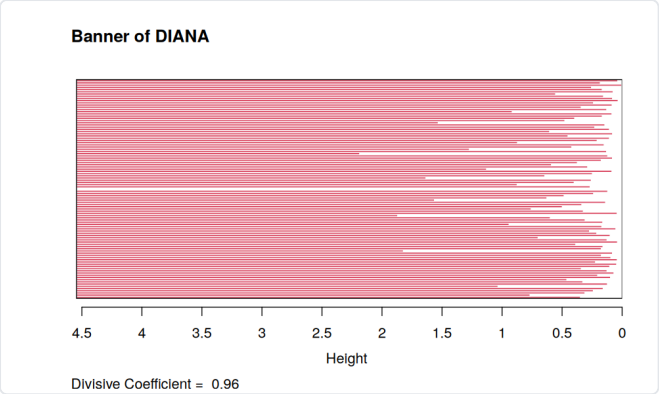
With single linkage the tree is "weird" and hard to read. Changing to complete linkage (and Manhattan distance) gives a nice tree. We can see **2 clusters** (ideally, since we know the data came from 2 populations), but a cut showing **4 clusters** is also acceptable. This trial and error is the normal way of doing cluster analysis.

> **Scale the data first**
> The lecturer noted that the first step should be to **scale (standardise)** the data – for agglomerative and divisive clustering (and k-means). Here the data are simulated on the same scale, but for real data with different units, use `scale()`. `hclust()` has no standardise option, so do it yourself: pre-processing (missing values etc.) → standardise → cluster.

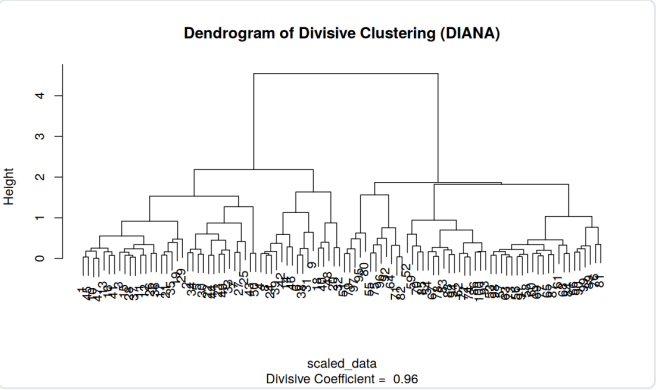
12. Divisive clustering with DIANA (top-down)

```
# top down approach (divisive hierarchical clustering)
scaled_data = scale(data)
library(cluster)
hc_div_out = diana(scaled_data)
plot(hc_div_out, main = "Dendrogram of Divisive Clustering (DIANA)")
```

- **DIANA** = DIvisive ANAlysis. The function `diana()` is in the **cluster** package (comes with R; just load it).
- You give the (scaled) data; nothing else is needed.
- The output contains the order of objects, heights and the **divisive coefficient** = **0.96** here (close to 1 = strong clustering structure).
- `plot()` gives two plots; press **Enter** in the console to see the next one: first a **banner** plot, then the **dendrogram**.
- Divisive starts with one cluster and divides it; agglomerative starts with n clusters and combines them.



Banner plot



DIANA dendrogram (divisive coefficient 0.96)

```
k_clusters = cutree(hc_div_out, k = 2)    # cut the tree into 2 clusters
table(k_clusters)                         # 50 50
```

`cutree()` gives the cluster number of each observation (like `$cluster` in k-means). It works for `hclust()` trees too. Here DIANA's 2 clusters match the k-means clusters exactly (50 / 50).

Nice plots with factoextra

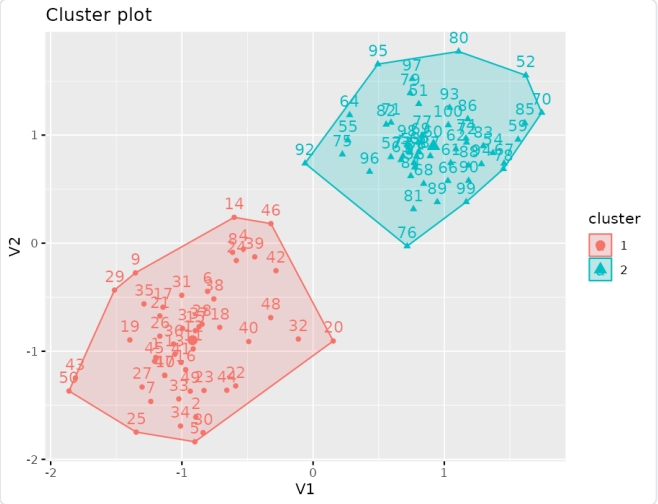
```
#install.packages("factoextra")   # only once
library(factoextra)
fviz_cluster(data = scaled_data, cluster = k_clusters)     # lecture code — ERROR
```

Fix for fviz_cluster()

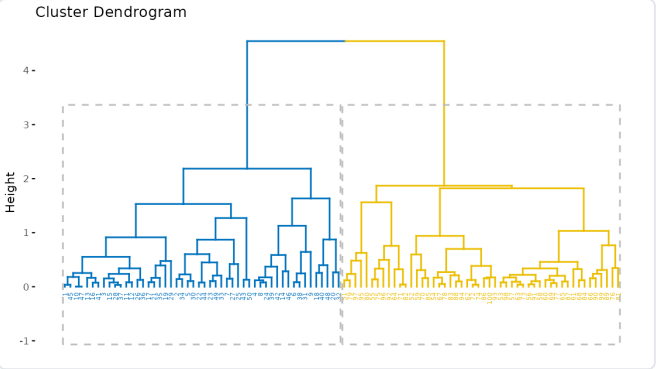
In class this line did not work ("argument "object" is missing"). `fviz_cluster()` needs a **list** with the data and the clusters, and the data must have **column names**:

```
colnames(scaled_data) <- c("V1", "V2")
fviz_cluster(list(data = scaled_data, cluster = k_clusters))
fviz_dend(hc_div_out, k = 2, rect = TRUE, cex = 0.4, palette = "jco")   # coloured dendrogram
```

You don't need to memorise these extra-package codes – look them up (e.g. the Datanovia website, which uses the USArrests data with `diana(USArrests, stand = TRUE)`).



fviz_cluster: 2 clusters



fviz_dend with 2 coloured clusters

13. Homework (due 05.01.2026 by 6 pm)

```
# HW: Due: 05.01.2026 by 6pm
# Use the USArrests dataset
data("USArrests")
# Carry out a cluster analysis.
# How many clusters do you see from
## 1. hierarchical agglomerative clustering with euclidean distance and complete linkage
## 2. divisive clustering
# confirm your choice above by using a K-means clustering.
```

- **USArrests**: one row per US state (Alabama, Alaska, …) with Murder, Assault, UrbanPop and Rape rates.
- Submit through the link that will be shared (LMS or a separate link) – **not** via WhatsApp or email.
- **Do not use AI tools.** The lecturer will give **zero marks** if the work is detected as AI-generated. So these notes do not give the answers – use the steps below and write your own code and interpretation.

Checklist of the methods you learnt (all shown in these notes)

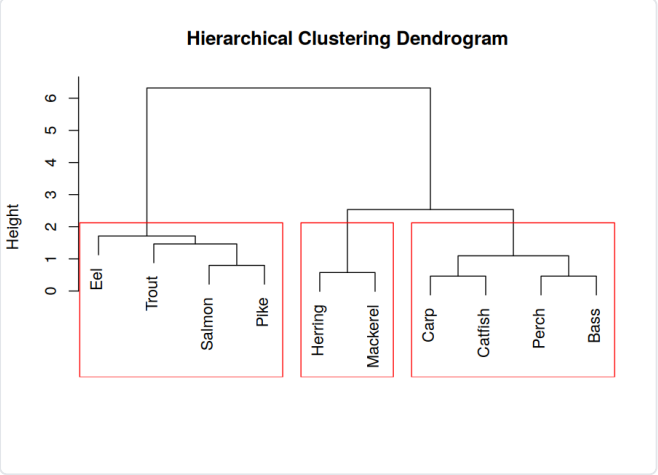
1. Look at the data (`head`, `str`, `summary`), check missing values, keep numeric variables.
2. **Scale** the data (the variables are in very different units).
3. Agglomerative: `dist(..., method = "euclidean")` → `hclust(..., method = "complete")` → dendrogram → cut where the vertical gap is largest → `rect.hclust()`, `cutree()`.
4. Divisive: `diana()` → dendrogram → `cutree()`.
5. k-means with your chosen k (`set.seed`, `nstart`), check between_SS / total_SS, compare cluster memberships with `table()`.
6. Explain in your own words why you chose that number of clusters.

14. Extra practice: fish data and penguins data

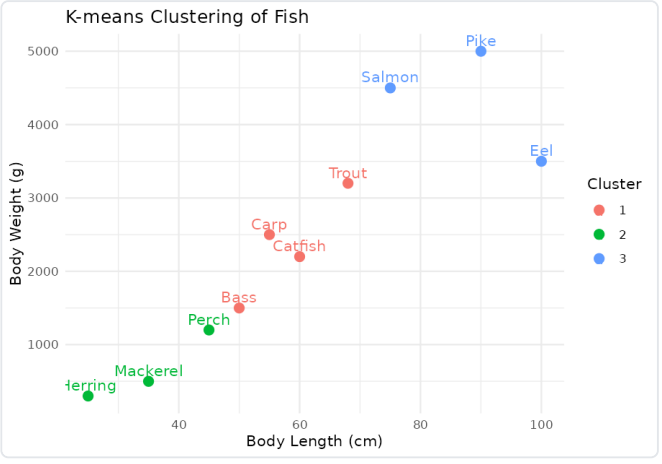
14.1 Fish (zoology) data – hierarchical vs k-means

```
fish_data <- data.frame(
  Species =
c("Salmon","Trout","Carp","Pike","Perch","Bass","Catfish","Eel","Herring","Mackerel"),
  Body_Length_cm = c(75, 68, 55, 90, 45, 50, 60, 100, 25, 35),
  Body_Weight_g  = c(4500, 3200, 2500, 5000, 1200, 1500, 2200, 3500, 300, 500),
  Fin_Length_cm  = c(20, 18, 15, 22, 12, 14, 13, 25, 8, 10))
fish_numeric <- fish_data[, -1]          # remove species column
fish_scaled  <- scale(fish_numeric)       # scale — weight (grams) would dominate otherwise
dist_matrix <- dist(fish_scaled, method = "euclidean")
hc <- hclust(dist_matrix, method = "ward.D2")
plot(hc, labels = fish_data$Species, main = "Hierarchical Clustering Dendrogram", xlab =
"", sub = "")
rect.hclust(hc, k = 3, border = "red")
```

- Scaling is essential here: weight is in thousands of grams, lengths in tens of cm. Without scaling, weight alone would decide the clusters.
- `labels = fish_data$Species` shows fish names instead of row numbers; `xlab = "", sub = ""` removes the text below the tree.



Ward.D2: {Eel, Trout, Salmon, Pike}, {Herring, Mackerel}, {Carp, Catfish, Perch, Bass}



k-means (k = 3) on the scaled data

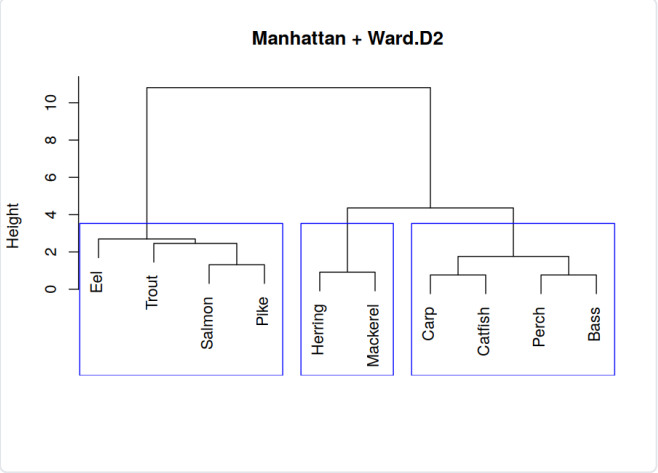
```
set.seed(123)                     # for reproducibility
kmeans_result <- kmeans(fish_scaled, centers = 3, nstart = 25)
fish_data$Cluster <- as.factor(kmeans_result$cluster)
fish_data
library(ggplot2)
ggplot(fish_data, aes(x = Body_Length_cm, y = Body_Weight_g, color = Cluster, label =
Species)) +
  geom_point(size = 3) + geom_text(vjust = -0.5) +
  labs(title = "K-means Clustering of Fish", x = "Body Length (cm)", y = "Body Weight (g)")
+
  theme_minimal()
# comparing methods
table(Hierarchical = cutree(hc, k = 3), KMeans = kmeans_result$cluster)
```

Method	Group A	Group B	Group C
Hierarchical (Ward)	Salmon, Trout, Pike, Eel	Carp, Perch, Bass, Catfish	Herring, Mackerel
k-means	Salmon, Pike, Eel	Trout, Carp, Bass, Catfish	Perch, Herring, Mackerel

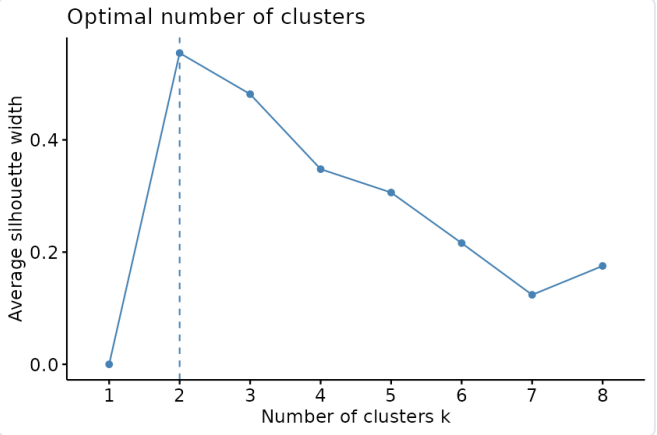
The `table()` shows the two methods **agree only partly** (Trout and Perch move). With only 10 fish, small differences in method change the groups – another reason to compare methods and use subject knowledge.

Exercises (from the R file)

1. Add more species and traits (e.g. tail length, scale count) to the data set.
2. Try a different distance (e.g. `"manhattan"`) in the hierarchical clustering – how does the dendrogram change?
3. Use the **NbClust** package to find the optimal number of clusters: `NbClust(fish_scaled, distance = "euclidean", min.nc = 2, max.nc = 6, method = "kmeans")` – it computes about 30 indices and suggests the k chosen by the majority. (A similar quick check: `fviz_nbclust(fish_scaled, kmeans, method = "silhouette")`.)
4. Apply clustering to another data set, such as iris.



Exercise 2: Manhattan + Ward.D2

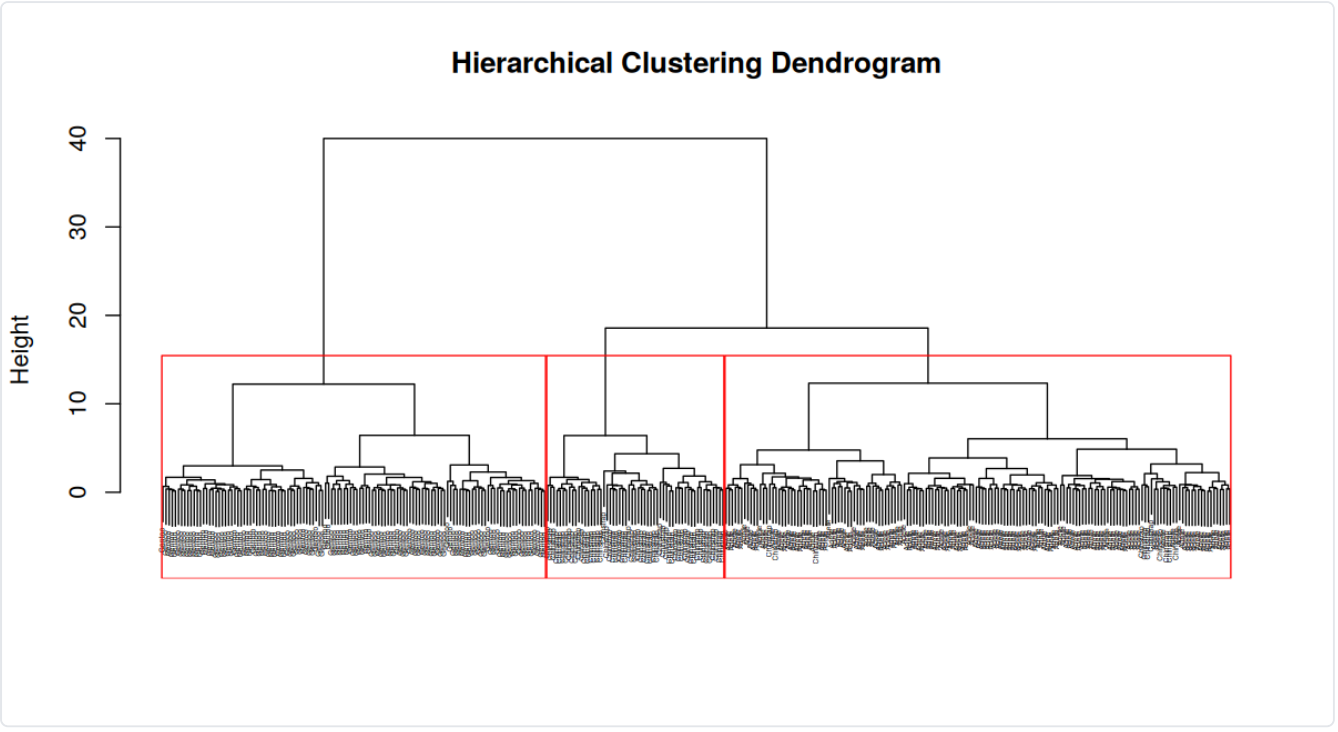


Exercise 3 idea: silhouette method

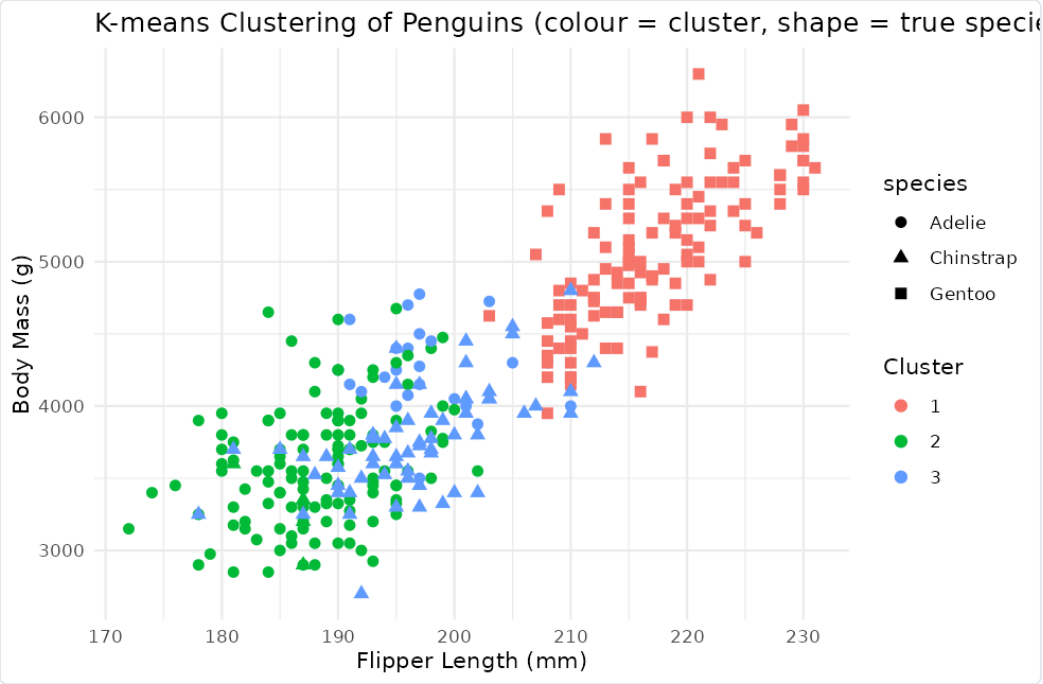
14.2 Palmer penguins

```
install.packages(c("palmerpenguins", "ggplot2", "NbClust"))   # once
library(palmerpenguins); library(ggplot2); library(NbClust)
head(penguins)
penguins_clean <- na.omit(penguins[, c("species", "bill_length_mm", "bill_depth_mm",
                                       "flipper_length_mm", "body_mass_g")])
penguins_numeric <- penguins_clean[, -1]          # numeric variables only
penguins_scaled  <- scale(penguins_numeric)
dist_matrix <- dist(penguins_scaled, method = "euclidean")
hc <- hclust(dist_matrix, method = "ward.D2")
plot(hc, labels = penguins_clean$species, main = "Hierarchical Clustering Dendrogram", xlab
= "", sub = "")
rect.hclust(hc, k = 3, border = "red")
set.seed(123)
kmeans_result <- kmeans(penguins_scaled, centers = 3, nstart = 25)
penguins_clean$Cluster <- as.factor(kmeans_result$cluster)
head(penguins_clean)
```

- `na.omit()` removes rows with missing values (344 → 342 penguins).
- 4 numeric measurements, 3 true species (Adelie, Chinstrap, Gentoo) – used only to check the result.



Ward.D2 dendrogram with 3 clusters



k-means clusters (the lecture's ggplot also writes species names on each point with geom_text; here the species is shown by shape for readability)

k-means cluster	Adelie	Chinstrap	Gentoo
1	0	0	123
2	127	5	0
3	24	63	0

Hierarchical (Ward) cluster	Adelie	Chinstrap	Gentoo
1	151	11	0
2	0	0	123
3	0	57	0

Both methods find **Gentoo perfectly** (large body, long flippers). Adelie and Chinstrap are similar in size and are partly mixed.

15. Summary

Key points

- Clustering groups **observations** so that members of a cluster are similar; it is **unsupervised** and an intermediate step (pre-process → cluster → model).
- Basic clustering needs **quantitative** data (distances). Make a copy, keep numeric columns, **scale**.
- **k-means:** choose k first; minimises within SS; check between_SS / total_SS; use `set.seed` and `nstart`; choose k with elbow/silhouette.
- **Hierarchical:** agglomerative (bottom-up, `hclust`) or divisive (top-down, `diana`); result = dendrogram; cut at the largest vertical gap.
- **Distance:** Euclidean $\sqrt{\Sigma(a_i - b_i)^2}$ (straight line) or Manhattan $\Sigma|a_i - b_i|$ (city blocks). `dist()` works on **rows**.
- **Linkage:** single (min), complete (max, default), average (mean), Ward (min increase in within SS).
- Different distances/linkages give different trees – try several and use domain knowledge.
- Useful functions: `kmeans`, `dist`, `hclust`, `plot`, `rect.hclust`, `cutree`, `diana`, `fviz_cluster`, `fviz_dend`, `fviz_nbclust`.

Test yourself

1. Why is clustering called unsupervised learning?
2. Why can't we compute a distance between "male" and "female"?
3. What does between_SS / total_SS = 88.4% mean?
4. Why might two runs of `kmeans()` give different results? How do you fix this?
5. Compute the Euclidean and Manhattan distances between (2, 3) and (5, 7). (Answer: 5 and 7)
6. Explain the difference between agglomerative and divisive clustering.
7. With single linkage, what is the distance between clusters {g1, g2} and {g3, g4} in Section 8? And with complete linkage? (Answer: 2.19 and 2.39)
8. How do you decide the number of clusters from a dendrogram?
9. Why must the observations be in the rows when you use `dist()`?
10. Why should you scale the fish data before clustering?